

CS6650 HW10

Sai Karthikeyan Sura
April 4th, 2026

How the Code Works

A single Go binary (`cmd/server/main.go`) runs in either leader-follower or leaderless mode based on environment variables (`MODE`, `STRATEGY`, `NODES`, `SELF_URL`, `LEADER_URL`). The in-memory store (`internal/kv/store.go`) is a `sync.RWMutex`-protected map with `Set`, `SetIfNewer`, and `Get` operations. Each node generates unique monotonic versions using nanosecond timestamps left-shifted by 8 bits, OR-ed with a node-specific FNV-1a hash suffix.

- **W5R1:** On write, the leader stores locally then replicates sequentially to all 4 followers (200ms sleep after each send, 100ms follower sleep before ack). Write returns only after all 4 ack. Reads return the leader's local value. Write latency: $\sim 1200\text{ms}$, read latency: $\sim 1\text{ms}$.
- **W1R5:** On write, the leader stores locally and returns 201 immediately. A background goroutine replicates asynchronously. On read, all 5 nodes are queried (followers sleep 50ms each) and the highest version is returned. Write latency: $\sim 1\text{ms}$, read latency: $\sim 210\text{ms}$.
- **W3R3:** On write, the leader stores locally (`ack=1`) then replicates sequentially until 3 acks. Remaining followers are replicated asynchronously via `replicateRemaining`, which skips already-acked nodes using a tracking map. On read, 3 random nodes are queried and the highest version returned. Quorum overlap ($W + R = 6 > N = 5$) guarantees freshness. Write latency: $\sim 600\text{ms}$, read latency: $\sim 100\text{--}160\text{ms}$.
- **Leaderless (W=N, R=1):** Any node becomes coordinator on write. It stores locally then replicates sequentially to all 4 peers (same delays). Returns 201 only after all ack. Reads return local state. Functionally equivalent to W5R1 latency but without a fixed leader.

Follower-to-leader proxying (`proxyToLeader`) forwards body and headers, returns 502 on failure. All inter-node HTTP calls use a 3-second timeout.

Consistency Testing

Three pytest tests verify consistency. All pass (Figure 1).

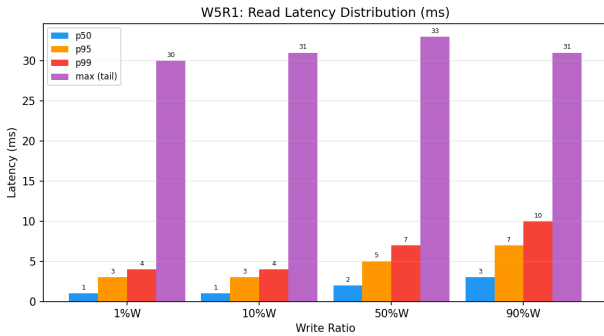
- `test_leader_consistency_after_ack`: Writes to leader, reads from leader and follower. Both return correct value.
- `test_leader_local_read_shows_inconsistency_window`: Writes to leader while polling `/local_read` on followers. Catches 404 during replication window.
- `test_leaderless_inconsistency_window_and_eventual_consistency`: Writes to random node, catches 404 on other nodes during fanout, then confirms consistency after ack.

Stale Reads

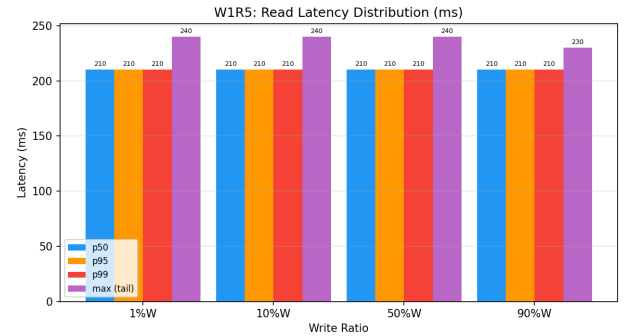
Config	W%	Versioned Reads	Stale Reads	Stale %
W5R1	1%	173,728	0	0.00%
W5R1	10%	34,257	0	0.00%
W5R1	50%	4,211	0	0.00%
W5R1	90%	446	0	0.00%
W1R5	1%	21,103	405	1.92%
W1R5	10%	22,785	4,155	18.24%
W1R5	50%	23,638	17,262	73.03%
W1R5	90%	15,444	13,308	86.17%
W3R3	1%	32,602	0	0.00%
W3R3	10%	24,515	0	0.00%
W3R3	50%	6,901	0	0.00%
W3R3	90%	903	0	0.00%
LL	1%	169,150	5,721	3.38%
LL	10%	33,665	1,995	5.93%
LL	50%	4,268	316	7.40%
LL	90%	454	34	7.49%

Table 2: Stale reads tracked by the client. “Versioned Reads” = reads where the key had a prior write version to compare against.

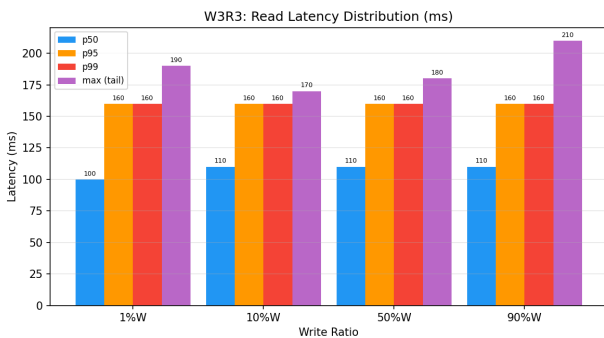
Read Latency Distribution



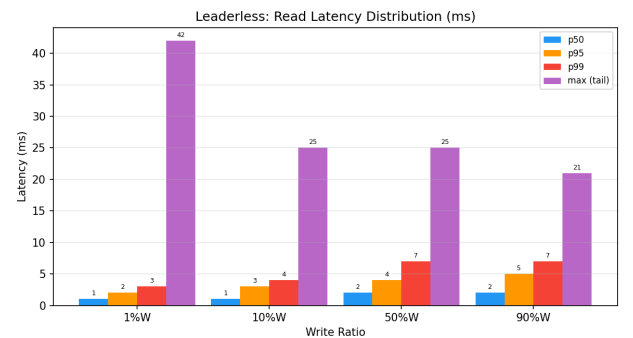
(a) W5R1



(b) W1R5



(c) W3R3

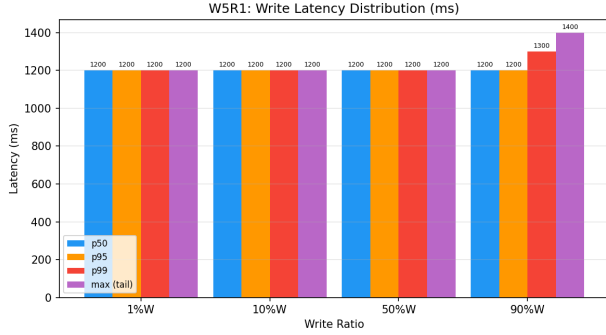


(d) Leaderless

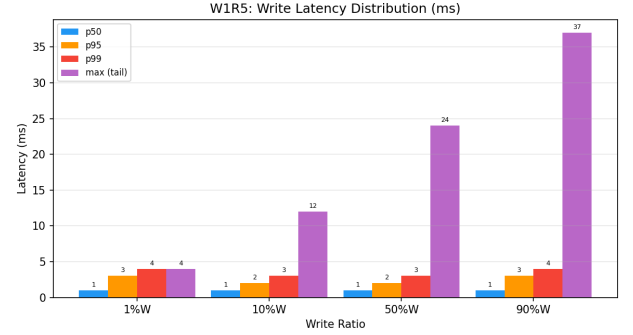
Figure 2: Read latency distribution (p50/p95/p99/max) across write ratios.

W5R1 and **Leaderless** show a long tail at 90% writes due to contention from concurrent writes blocking the server. **W1R5** reads are consistently $\sim 210\text{ms}$ regardless of the write ratio, since every read must query all 5 nodes and wait for the slowest responses. **W3R3** reads sit slightly lower at $\sim 100\text{--}160\text{ms}$, benefiting from only needing to query a quorum of 3 nodes.

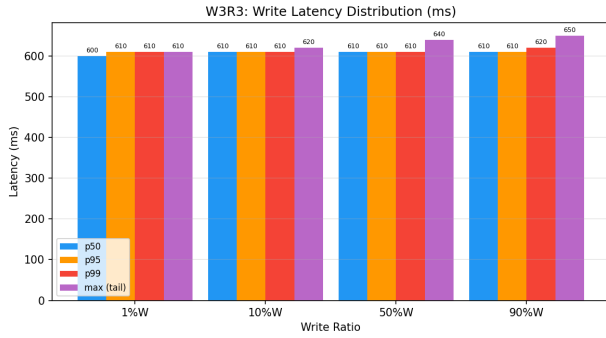
Write Latency Distribution



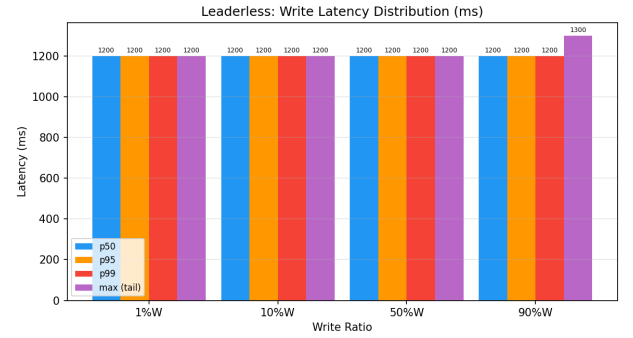
(a) W5R1



(b) W1R5



(c) W3R3

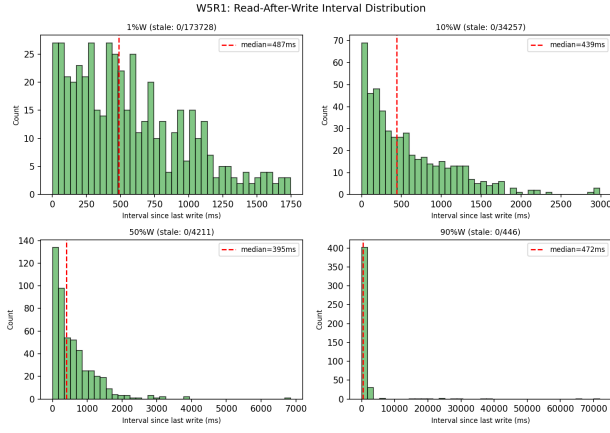


(d) Leaderless

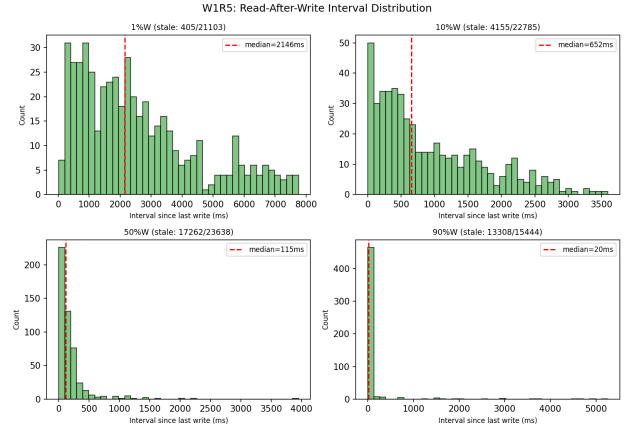
Figure 3: Write latency distribution.

W5R1 and **Leaderless** are locked at $\sim 1200\text{ms}$ (4 sequential replications $\times$ 300ms each), with a tail reaching $\sim 1400\text{ms}$ at 90% writes under contention. **W1R5** writes are $\sim 1\text{ms}$ (leader-only). **W3R3** writes are $\sim 600\text{ms}$ (2 follower acks before return).

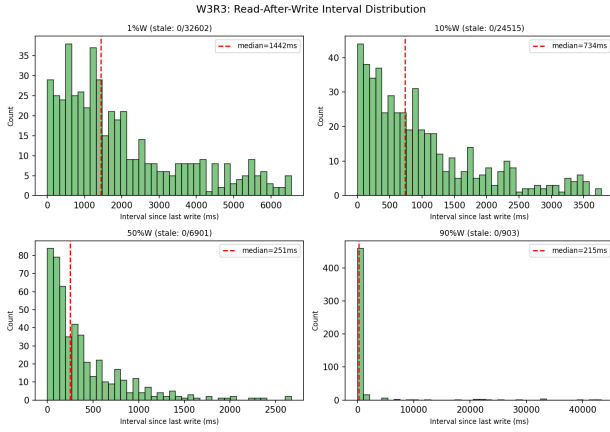
Read-Write Interval Distribution



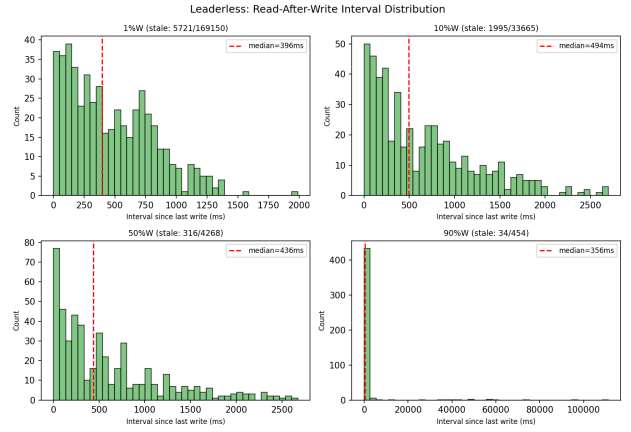
(a) W5R1



(b) W1R5



(c) W3R3



(d) Leaderless

The hot-key workload successfully concentrates the time between writing and reading the same key in the 10–500ms range (visible as the dense clustering between 0.0 and 0.5 seconds on the x-axis). For **W1R5**, reads that land within the ~1200ms asynchronous replication window hit un-updated nodes, producing massive stale reads (up to 17,262 at 50% writes). For **Leaderless** ($R = 1$), these rapid follow-up reads hit replicas before the Write Coordinator’s concurrent fanout finishes, also producing stale reads (5,721 at 1% writes). As expected by quorum theory, **W5R1** and **W3R3** strictly enforce consistency and show zero stale reads regardless of the interval length.

Stale Reads Summary

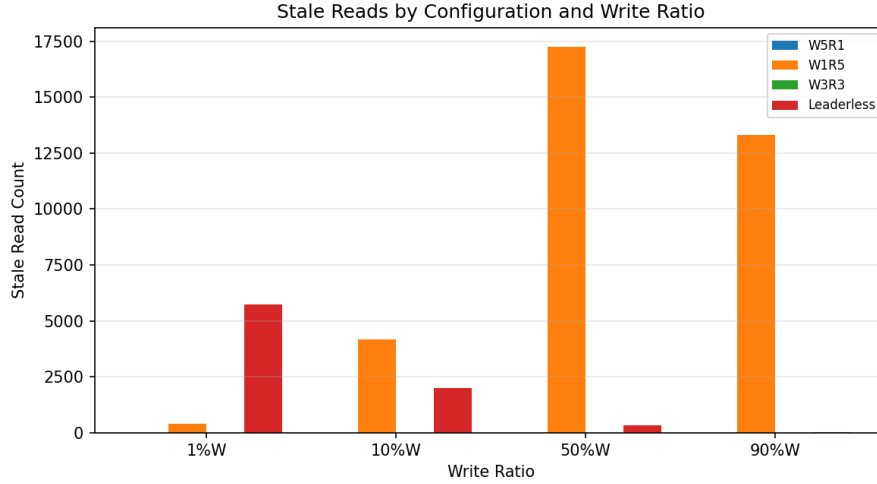


Figure 5: Stale reads across all configurations.

W5R1 and **W3R3** strictly guarantee consistency, showing zero stale reads across all workloads. For **W1R5**, stale reads increase significantly as the write ratio grows (peaking at 50% writes), because more frequent writes create more opportunities for reads to land in the asynchronous replication window. Conversely, **Leaderless** stale reads are highest at low write ratios (e.g., 1% writes). This occurs because the massive volume of concurrent reads are highly likely to hit replicas while the Write Coordinator is still processing its simulated fanout delays.

Analysis

Which Strategy Performs Best per Ratio?

- **Read-heavy (1% writes):** W5R1 at 1,708 RPS with 1ms reads. The rare 1200ms writes barely affect throughput.
- **Balanced (50/50):** W1R5 at 409 RPS. Its 1ms writes and 210ms reads average better than W5R1's 1200ms writes (79 RPS). W3R3 sits at 128 RPS.
- **Write-heavy (90% writes):** W1R5 at 1,296 RPS. 90% of operations complete in 1ms. W5R1 collapses to 44 RPS.

Why These Results?

Each sequential follower replication costs $\sim 300\text{ms}$ (100ms follower sleep + 200ms leader sleep). W5R1: $4 \times 300 = 1200\text{ms}$ writes. W3R3: $2 \times 300 = 600\text{ms}$ writes. W1R5 pushes cost to reads: $4 \times 50 = 200\text{ms}$ follower delay plus HTTP overhead.

Throughput follows:

$$\text{RPS} \approx \frac{\text{Users}}{\text{write_ratio} \times T_w + \text{read_ratio} \times T_r}$$

This explains why W1R5 throughput increases with write ratio ($T_w = 1\text{ms} \ll T_r = 210\text{ms}$).

Stale Read Behavior

W5R1 has zero stale reads: $W=N$ blocks until all replicas are updated. W3R3 also has zero: $W + R = 6 > N = 5$ guarantees quorum overlap, and the read path returns the highest version. W1R5 stale reads reach 86% at 90% writes. Writes return after updating only the leader while async replication takes $\sim 1200\text{ms}$. With high write throughput, newer versions are constantly being written while older versions haven't finished propagating, so most reads see a version behind the latest the client has observed.

Leaderless stale reads are 3–7%. Writes block until $W=N$, so the staleness window is only the $\sim 1200\text{ms}$ during sequential fanout. A concurrent $R=1$ read to an unreplicated node during this window returns stale data.

Best Architecture per Application

- **Read-heavy (social media, caching):** W5R1 or Leaderless. Sub-millisecond reads, zero API-visible stale reads.
- **Balanced (shopping carts, session state):** W3R3. Moderate latency on both operations, zero stale reads.
- **Write-heavy (logging, telemetry):** W1R5. Near-instant writes, stale reads acceptable for data rarely queried in real-time.
- **Strong consistency (financial transactions):** W5R1. Higher write latency is acceptable when correctness matters.